

Lab 4 – Breakout

Chat / LLM History – Clean Submission Copy

Topic: #10 – Breakout

Repository: aarush-170/Lab4_10_breakout (student fork)

Purpose: A readable record of the LLM-assisted debugging, implementation, testing, Git workflow, and submission preparation for the Breakout lab.

Note: This is a cleaned, submission-friendly reconstruction of the relevant conversation rather than a browser printout of the ChatGPT interface. It preserves the substantive questions, instructions, code changes, testing observations, and Git milestones from the attached exported chat and this conversation.

1. Assignment and Initial Plan

User: “my topic is the 10th one in the list, guide me step wise”

Assistant: Identified topic #10 as Breakout and broke the lab into four required tasks:

Task	Requirement
1. Fix bug	Remove a brick when hits_remaining reaches 0.
2. Lives	3 lives, lose a life when the ball falls, reset ball, Game Over, restart with R.
3. Brick types	Normal, Strong, and Unbreakable bricks with distinct behavior/appearance.
4. Combo scoring	Score system and consecutive-hit combo multiplier; missing the ball resets the combo.

The original game was run before modifying code, and the required before recording was made to capture the deliberate brick-removal bug.

2. Repository Setup

User: Shared the forked repository: [aarush-170/Lab4_10_breakout](#).

Assistant: Confirmed that work should be pushed to the student's fork rather than the original repository, and that no pull request should be raised to the main SETAPESU26 repository.

```
git clone https://github.com/aarush-170/Lab4_10_breakout.git
cd Lab4_10_breakout
dir
```

The original game was then run successfully.

3. Before Version

User: Showed the running original Breakout game and asked whether to record it.

Assistant: Confirmed that the original gameplay should be recorded as the BEFORE video, with the deliberate bug visible. The brick's hits_remaining could decrease, but the brick was not removed.

4. Task 1 + Task 2 – Brick Removal and Lives

User: Shared the original game_engine.py and requested the entire corrected file.

Assistant: Added brick-removal logic and the 3-life system.

```
# Remove a brick when its hit count reaches zero.
brick.hits_remaining -= 1
if brick.hits_remaining <= 0:
    self.bricks.remove(brick)

STARTING_LIVES = 3

self.lives = STARTING_LIVES
self.game_over = False

# When the ball is lost:
self.lives -= 1

if self.lives > 0:
    self._reset_ball()
else:
    self.game_over = True
```

Restart: pressing R after Game Over calls _restart_game(), restoring the paddle, ball, lives, game state, and bricks.

Testing showed a Game Over screen with Lives: 0 and the message “GAME OVER - Press R to Restart”. Restarting restored the game to 3 lives.

5. Git Milestone – Tasks 1 and 2

```
git add game/game_engine.py
```

```
git commit -m "Complete Tasks 1 and 2: brick removal and lives"
git push origin main
```

Recorded commit: 97a482b – Complete Tasks 1 and 2: brick removal and lives.

6. Task 3 – Brick Types

User: Shared the Brick class and implemented the three required brick types.

```
NORMAL = "normal"
STRONG = "strong"
UNBREAKABLE = "unbreakable"
```

Brick rows were assigned different types:

```
if row == 0:
    brick_type = Brick.UNBREAKABLE
elif row == 1:
    brick_type = Brick.STRONG
else:
    brick_type = Brick.NORMAL
```

Unbreakable bricks are never destroyed; normal and strong bricks lose one hit at a time.

```
if brick.brick_type == Brick.UNBREAKABLE:
    break

brick.hits_remaining -= 1

if brick.hits_remaining <= 0:
    self.bricks.remove(brick)
```

The resulting game showed visually distinct gray, orange, and red brick rows.

7. Git Milestone – Task 3

```
git log --oneline --all

29c736b Complete Task 3: add brick types
97a482b Complete Tasks 1 and 2: brick removal and lives
1382000 Initial Breakout starter
```

The Task 3 commit was pushed to the student's fork.

8. Task 4 – Score and Combo

User: Asked for the entire corrected Task 4 file.

Assistant: Implemented score and combo tracking while preserving Tasks 1–3.

```
NORMAL_SCORE = 10
STRONG_SCORE = 20

self.score = 0
self.combo = 0
```

For breakable bricks, the combo increases on each successful hit. When a brick is destroyed, its base score is multiplied by the current combo.

```
self.combo += 1
brick.hits_remaining -= 1

if brick.hits_remaining <= 0:
    if brick.brick_type == Brick.NORMAL:
        self.score += NORMAL_SCORE * self.combo
    elif brick.brick_type == Brick.STRONG:
        self.score += STRONG_SCORE * self.combo

    self.bricks.remove(brick)
```

Unbreakable bricks do not increase score or combo. Losing the ball resets the combo to zero while preserving accumulated score.

```
if self.ball.is_below(HEIGHT):
    self.lives -= 1
    self.combo = 0

    if self.lives > 0:
        self._reset_ball()
    else:
```

```
self.game_over = True
```

9. Task 4 HUD and Testing

The HUD was updated to display bricks remaining, score, combo multiplier, and lives.

```
renderer.draw_text(surface, font, f"Bricks left: {len(self.bricks)}", (10, 10))
renderer.draw_text(surface, font, f"Score: {self.score}", (10, 35))
renderer.draw_text(surface, font, f"Combo: x{self.combo}", (10, 60))
renderer.draw_text(surface, font, f"Lives: {self.lives}", (10, 85))
```

Testing result: the final gameplay screenshot showed Score: 370, Combo: x0 after losing the ball, Lives: 0, and the Game Over banner. This confirmed that score tracking worked and that the combo reset after the ball was lost.

10. Final Gameplay Recording Plan

Stage	Demonstration
Start	Bricks left, Score, Combo, Lives visible.
Gameplay	Move paddle and hit breakable bricks.
Strong brick	Show the orange strong brick requiring multiple hits.
Unbreakable	Show the gray unbreakable brick remaining.
Combo	Show score/combo changing after consecutive hits.
Miss	Show lives decreasing and combo returning to x0.
Game Over	Show Lives: 0 and GAME OVER – Press R to Restart.
Restart	Press R and show Score: 0, Combo: x0, Lives: 3 and a fresh layout.

11. Submission Deliverables

The deliverables shown in the conversation specified:

- Videos (before and after)
- Updated Code
- Chat history exported as a doc/pdf

The agreed folder structure is:

```
Lab4_10_breakout/
├── breakout/
│   ├── game/
│   └── videos/
│       ├── before.mp4
│       └── after.mp4
├── main.py
├── requirements.txt
└── Chat_History_Lab4_Breakout.pdf
```

12. Git / Repository Policy

The conversation explicitly checked whether work was being pushed to the original repository. The workflow used the student's fork, aarush-170/Lab4_10_breakout, and did not raise a pull request to the original SETAPESU26 repository.

Known commit history:

```
29c736b Complete Task 3: add brick types
97a482b Complete Tasks 1 and 2: brick removal and lives
1382000 Initial Breakout starter
```

Task 4 commit: Task 4 was tested successfully in the conversation, but a final Task 4 commit was not confirmed in the recorded chat. It should be committed after the final working-tree check.

13. Final Checklist

- ✓ Task 1 – brick removal
- ✓ Task 2 – 3 lives, ball reset, Game Over, restart
- ✓ Task 3 – Normal, Strong, Unbreakable bricks
- ✓ Task 4 – score, combo multiplier, HUD display, combo reset on miss
- ✓ Before gameplay recorded
- ✓ After gameplay tested
- ✓ Changes pushed to the student's fork
- ✓ No PR raised to the original repository

Remaining submission actions:

1. Ensure before.mp4 and after.mp4 are inside videos/.
2. Place this PDF in the Lab-4 folder if the submission is repository-based.
3. Verify final Task 4 Git status and create the final commit if required.
4. Push the final commit to the student fork.

Appendix A – Final game_engine.py used for Task 4

This is the final Task 4 version supplied and tested during the conversation.

```
"""
GameEngine: owns the paddle, ball, and bricks.

Task 1:
- Bricks are removed when their hits_remaining reaches 0.

Task 2:
- Player starts with 3 lives.
- Losing the ball costs one life.
- Ball resets after losing a life.
- Game over occurs when all lives are lost.
- Press R to restart after game over.

Task 3:
- Normal bricks require 1 hit.
- Strong bricks require multiple hits.
- Unbreakable bricks can never be destroyed.

Task 4:
- Normal bricks give 10 base points.
- Strong bricks give 20 base points.
- Consecutive successful hits increase the combo multiplier.
- Score earned is multiplied by the current combo.
- Combo resets when the ball is lost.
- Score and combo are displayed.
"""

import pygame

from game.paddle import Paddle
from game.ball import Ball
from game.brick import Brick
from game.collision import handle_ball_brick_collision
from game.renderer import WIDTH, HEIGHT

BRICK_ROWS = 4
BRICK_COLS = 8
BRICK_WIDTH = 68
BRICK_HEIGHT = 22
BRICK_GAP = 6
BRICK_TOP_MARGIN = 50

STARTING_LIVES = 3

NORMAL_SCORE = 10
STRONG_SCORE = 20

class GameEngine:
    def __init__(self):
        self.paddle = Paddle(x=WIDTH / 2, y=HEIGHT - 30)
        self.ball = Ball(x=WIDTH / 2, y=HEIGHT - 50)

        self.lives = STARTING_LIVES
        self.game_over = False

        self.score = 0
        self.combo = 0

        self.bricks = self._build_bricks()

    def _build_bricks(self):
        bricks = []

        total_width = BRICK_COLS * (BRICK_WIDTH + BRICK_GAP) - BRICK_GAP
        start_x = (WIDTH - total_width) / 2

        for row in range(BRICK_ROWS):
            for col in range(BRICK_COLS):
                x = start_x + col * (BRICK_WIDTH + BRICK_GAP)
                y = BRICK_TOP_MARGIN + row * (BRICK_HEIGHT + BRICK_GAP)

                if row == 0:
                    brick_type = Brick.UNBREAKABLE
                elif row == 1:
                    brick_type = Brick.STRONG
                else:
                    brick_type = Brick.NORMAL

                bricks.append(
                    Brick(
                        x,

```

```

        y,
        BRICK_WIDTH,
        BRICK_HEIGHT,
        brick_type=brick_type,
    )
)

return bricks

def _reset_ball(self):
    self.ball = Ball(x=WIDTH / 2, y=HEIGHT - 50)

def _restart_game(self):
    self.paddle = Paddle(x=WIDTH / 2, y=HEIGHT - 30)
    self.ball = Ball(x=WIDTH / 2, y=HEIGHT - 50)

    self.lives = STARTING_LIVES
    self.game_over = False

    self.score = 0
    self.combo = 0

    self.bricks = self._build_bricks()

def handle_input(self, keys_pressed):
    if self.game_over:
        return

    dx = 0

    if keys_pressed[pygame.K_LEFT]:
        dx -= self.paddle.speed

    if keys_pressed[pygame.K_RIGHT]:
        dx += self.paddle.speed

    self.paddle.move(dx, WIDTH)

def handle_keydown(self, key):
    if self.game_over and key == pygame.K_r:
        self._restart_game()

def update(self):
    if self.game_over:
        return

    self.ball.update()
    self.ball.bounce_off_walls(WIDTH)

    if (
        self.ball.get_rect().collidect(self.paddle.get_rect())
        and self.ball.vy > 0
    ):
        self.ball.bounce_off_paddle(self.paddle.get_rect())

    for brick in self.bricks:
        if handle_ball_brick_collision(self.ball, brick):

            if brick.brick_type == Brick.UNBREAKABLE:
                break

            self.combo += 1
            brick.hits_remaining -= 1

            if brick.hits_remaining <= 0:

                if brick.brick_type == Brick.NORMAL:
                    self.score += NORMAL_SCORE * self.combo

                elif brick.brick_type == Brick.STRONG:
                    self.score += STRONG_SCORE * self.combo

                self.bricks.remove(brick)

            break

    if self.ball.is_below(HEIGHT):
        self.lives -= 1
        self.combo = 0

        if self.lives > 0:
            self._reset_ball()
        else:
            self.game_over = True

def draw(self, surface, font):
    from game import renderer

```

```

    renderer.draw_scene(
        surface,
        self.paddle,
        self.ball,
        self.bricks
    )

    renderer.draw_text(
        surface,
        font,
        f"Bricks left: {len(self.bricks)}",
        (10, 10)
    )

    renderer.draw_text(
        surface,
        font,
        f"Score: {self.score}",
        (10, 35)
    )

    renderer.draw_text(
        surface,
        font,
        f"Combo: x{self.combo}",
        (10, 60)
    )

    renderer.draw_text(
        surface,
        font,
        f"Lives: {self.lives}",
        (10, 85)
    )

    if self.game_over:
        renderer.draw_banner(
            surface,
            font,
            "GAME OVER - Press R to Restart"
        )

```